Lab 1 Report

Authors:

Aaron Tianle Shi, [22]

Date: October 18, 2026

Course #: EE P 535

Introduction 3
System Design and Implementation 3
Discussion 3
Total Number of hours Spent 3
References 4

Introduction

Purpose. The purpose of Lab 1 was to take the digital logic concepts from lecture and make them run on a real device. Concretely, the lab walks through the complete FPGA development flow on the Terasic DE1-SoC board: writing SystemVerilog in Quartus Prime Lite, verifying behaviour in ModelSim, compiling the design into a bitfile, and downloading it to the Cyclone V FPGA. The lab is organised in three parts — becoming familiar with the board and the tool chain, practising combinational logic design, and finally describing behaviour at the register-transfer level (RTL) so that the CAD tool performs the Boolean optimisation.

Key concepts. SystemVerilog modules and ports; structural instantiation of submodules; testbenches with initial blocks, #10 delays and for loops, used only for simulation and never loaded onto the FPGA; the distinction between simulation and synthesis; hand optimisation of combinational logic with truth tables and Karnaugh maps; RTL description with always_comb and case; active-low seven-segment encoding; and don't-care conditions and how they are resolved into fixed values in hardware.

Learning outcomes. Build and simulate a hierarchical design by reusing an already-verified small module; read ModelSim waveforms including unknown (red, X) and disconnected (blue, Z) signals; design a combinational circuit by hand and count the gates it costs; write RTL that describes intended behaviour instead of gate-level structure; and complete the full loop from source code to a working circuit on the board.

System Design and Implementation

All five tasks follow the same seven-step loop: copy the previous project folder, write the module and its testbench, set the top-level entity, run Analysis & Synthesis, simulate in ModelSim, compile to a bitfile, and test on the board. Each task lives in its own directory (lab1-1.3, lab1-2.1, lab1-2.2, lab1-3.1, lab1-3.2) so that a later task can reuse an earlier, already-working design as a starting point. This section describes each design and the reasoning behind it.

2.1 Task 1.3 — 4:1 Multiplexer

A 4:1 multiplexer was built from three copies of the 2:1 multiplexer developed in the tutorial. The lower select bit sel0 chooses one input from each of the two pairs, producing the internal wires v0 and v1; the upper select bit sel1 then decides which of those two results reaches out. As a result, the two-bit value "sel1 sel0" directly names the data input routed to the output.

```
module mux4_1(out, i00, i01, i10, i11, sel0, sel1);
  output logic out;
  input  logic i00, i01, i10, i11, sel0, sel1;
  logic  v0, v1;
  mux2_1 m0(.out(v0),  .i0(i00), .i1(i01), .sel(sel0));
  mux2_1 m1(.out(v1),  .i0(i10), .i1(i11), .sel(sel0));
  mux2_1 m (.out(out), .i0(v0),  .i1(v1),  .sel(sel1));
endmodule
```

The testbench sweeps all 64 combinations of the four data inputs and the two select bits ({sel1, sel0, i00, i01, i10, i11} = i; #10), so every select value is exercised against every data pattern. In plain English, the circuit accomplishes the following.

The mux4_1 is a 4-to-1 multiplexer: it uses two select bits, sel1 and sel0, to choose which one of its four data inputs (i00, i01, i10, i11) appears on the single output out. The lower select bit sel0 first picks one input from each of the two pairs, and the upper select bit sel1 then picks which of those two results drives out, so the two-bit value "sel1 sel0" directly names the chosen input. It is built from three copies of the mux2_1 module, which means a wider multiplexer is created by reusing a small, already-verified block rather than by writing new Boolean equations.

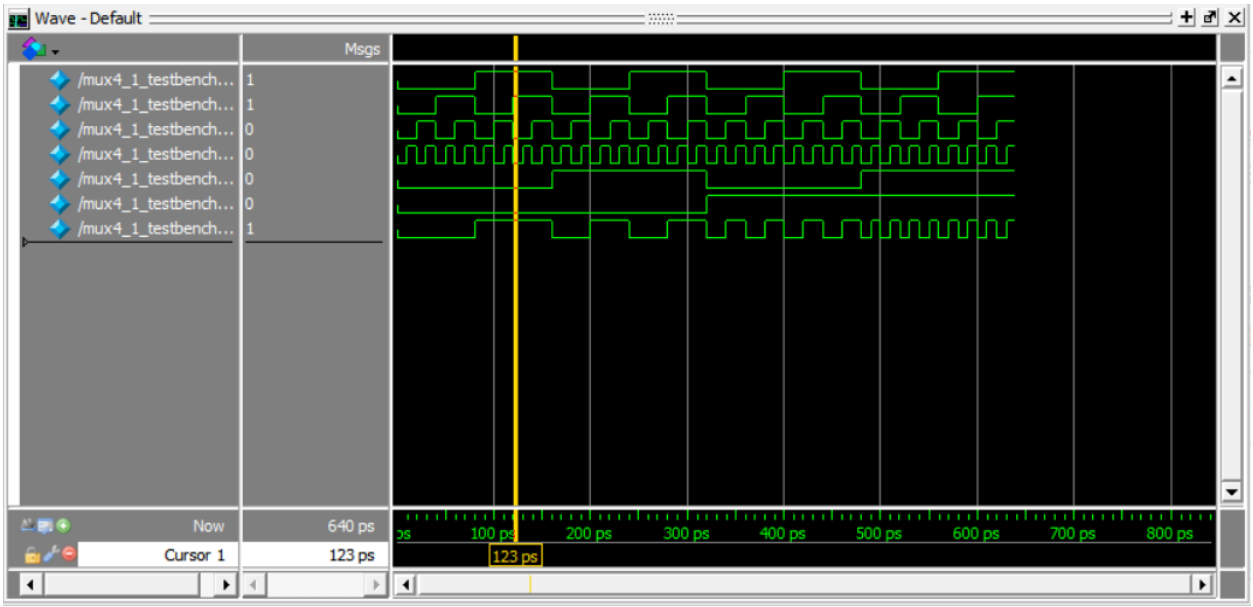


Figure 1. ModelSim waveform of mux4_1 (all 64 input combinations).

2.2 Task 2.1 — Two-Digit Recognizer

This task first puts the 2:1 multiplexer onto the board through the top-level module DE1_SoC, then replaces that logic with a recognizer for the last two digits of my student ID. SW[3:0] encodes the ones digit and SW[7:4] the tens digit; LEDR[0] turns on only when both digits match. My two digits are 22, and since 2 is 4'b0010, both comparisons use the same constant.

```
// Default values: turn off the HEX displays (active-low, all 1's = blank)
assign HEX0 = '1;   ...   assign HEX5 = '1;

// LEDR[0] = 1 only when SW[7:4] matches the tens digit AND SW[3:0] matches the ones digit
assign LEDR[0] = (SW[7:4] == 4'b0010) && (SW[3:0] == 4'b0010);

// Equivalent gate-level form:
// wire tens_ok = ~SW[7] & ~SW[6] &  SW[5] & ~SW[4];
// wire ones_ok = ~SW[3] & ~SW[2] &  SW[1] & ~SW[0];
// assign LEDR[0] = tens_ok & ones_ok;
```

On the board this means SW5 and SW1 must be up and every other switch down. The testbench sweeps all 256 combinations of SW[7:0]; the matching combination is 0b0010_0010

= 34, and the simulation confirms that LEDR[0] rises for exactly that one pattern and stays low for the other 255. The testbench lives in the same .sv file as the design; because DE1_SoC never instantiates it, none of its code is loaded onto the FPGA and it does not need to be commented out.



Figure 2. ModelSim waveform of the two-digit recognizer (256 combinations, LEDR[0] high once).

2.3 Task 2.2 — UPC Return Detector

A shop wants a circuit that flags discounted items and detects stolen goods. Six products exist, so only six of the eight UPC codes are used and the remaining two are don't cares. The inputs are U = SW9, P = SW8, C = SW7 (the UPC) and M = SW0 (secret mark); the outputs are LEDR[1] for Discounted and LEDR[0] for Stolen. An expensive item sold legitimately carries the secret mark, so a returned expensive item without the mark was stolen.

2.3.1 Truth table and minimisation

Discounted depends only on the UPC; Stolen is 1 only when an expensive item is returned without the mark (M = 0). Discounted is 1 for 011, 101 and 110, 0 for 000, 001 and 100, and X for 010 and 111.

Discounted: the three 1-cells are not adjacent to each other, but merging them with the don't-care cell 111 gives two terms — P, which covers 011 and 110, and U·C, which covers 101. No 0-cell is covered. This costs one two-input AND plus one two-input OR, i.e. 2 gates, which is minimal because the function is neither a constant nor a single literal.

Stolen: the expensive items are 000, 100 and 101. Minimising the four-variable function directly, rather than computing a separate "expensive" signal and multiplying it by M', saves one gate: the term NOR(P, C, M) covers 000 and 100 when M = 0, and the term U·C·M' covers 101 when M = 0.

Discounted = P + (U AND C) → **AND2 + OR2 = 2 gates**

Stolen = NOR(P, C, M) OR (U AND C AND NOT M) → **NOR3 + NOT + AND3 + OR2 = 4 gates**

The total is 6 gates, the minimum for this pair of functions. Computing E = (P NOR C) OR (U AND C) first and then taking E AND NOT M would cost 5 gates instead of 4, so the direct form was used.

2.3.2 Implementation and results

```
assign LEDR[1] = P | (U & C);                  // Discounted, 2 gates
assign LEDR[0] = ~(P | C | M) | (U & C & ~M);  // Stolen, 4 gates
```

Because 010 and 111 were specified as X, the synthesis tool was free to assign them; on the board, 010 turns on the Discounted LED, and 111 turns on both LEDs when M = 0. These extra cases are legal because the two codes never occur for real products, but they show that a don't-care condition becomes a fixed value in hardware rather than staying undefined.

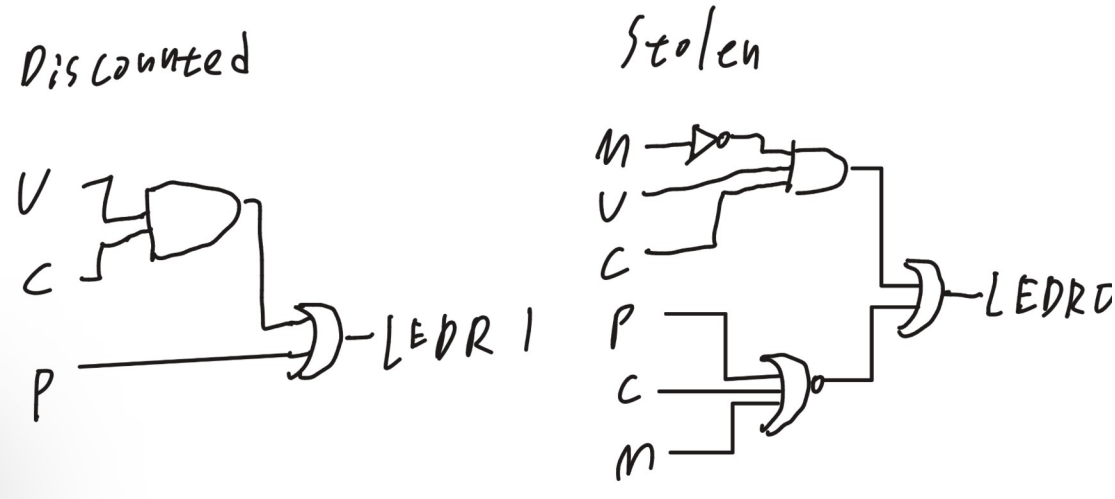


Figure 3. Gate-level schematic of the UPC detector (6 gates total).

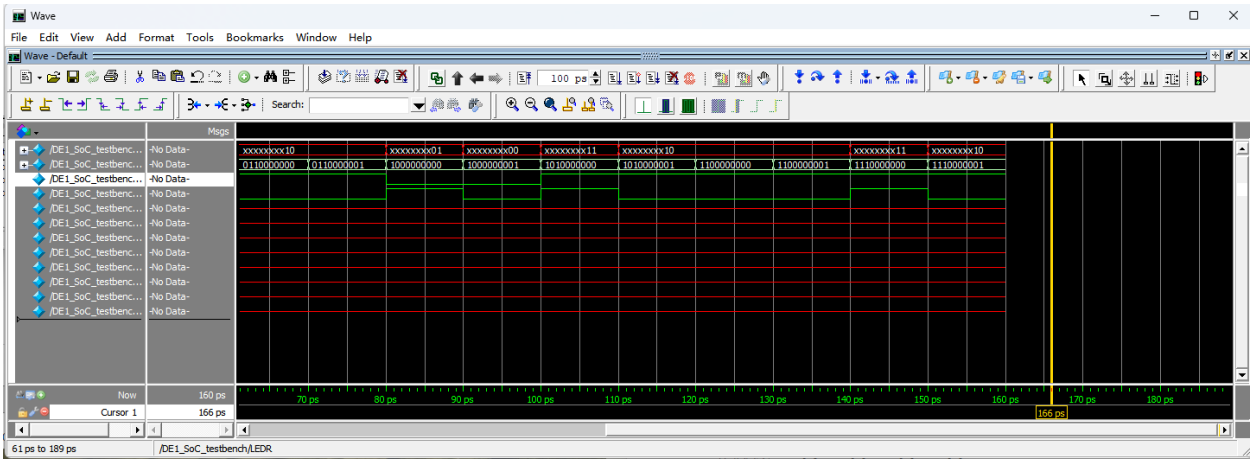


Figure 4. ModelSim waveform of the UPC detector (all 16 combinations of U, P, C and M).

2.4 Task 3.1 — Two Seven-Segment Displays

The seven-segment decoder seg7 from the handout translates a 4-bit BCD value into seven segment bits where 1 means "lit". The DE1-SoC segments, however, are active-low — a segment lights when its bit is 0 — so the top level must invert the decoder output. Two copies of seg7 are instantiated, one driven by SW[3:0] and one by SW[7:4].

```
seg7 d0 (.bcd(SW[3:0]), .leds(ones));    // ones digit -> HEX0
seg7 d1 (.bcd(SW[7:4]), .leds(tens));    // tens digit -> HEX1

assign HEX0 = ~ones;    // active-low
assign HEX1 = ~tens;
assign HEX2 = 7'b1111111;  ...  assign HEX5 = 7'b1111111;   // unused displays off
assign LEDR  = 10'b0;
```

Simulated values confirm the encoding: 0 → 1000000 (only segment g off), 1 → 1111001, 8 → 0000000 (all segments on). Inputs 10–15 fall through to the default branch (7'bX) and were observed on the board to display as a mirrored 9, 9, 9, 5, 6 and 9 respectively, because each illegal code is an independent don't care.

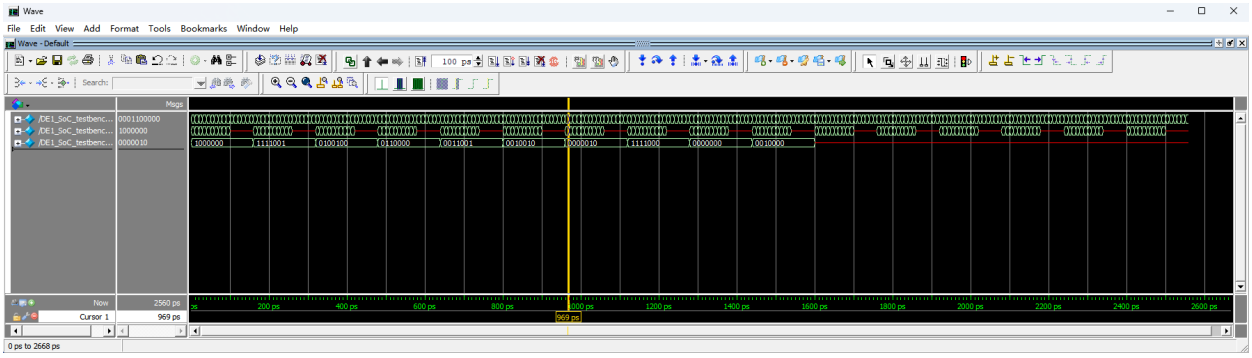


Figure 5. ModelSim waveform of the two-digit seven-segment display (HEX1 and HEX0).

2.5 Task 3.2 — UPC Code to Display

This task adds a display that names the product whose UPC is entered, so a swapped sticker is easy to spot. The display module has three inputs (U, P, C) and six 7-bit outputs, one per display, and is written at the RTL level with always_comb and case. Richer products must keep the UPC codes marked "expensive" (000, 100, 101), while the inexpensive ones use 001, 011 and 110.

```
upc_display disp (.U(U), .P(P), .C(C),
                  .h5(HEX5), .h4(HEX4), .h3(HEX3),
                  .h2(HEX2), .h1(HEX1), .h0(HEX0));

assign LEDR[1] = P | (U & C);                  // Discounted (same as Task 2.2)
assign LEDR[0] = ~(P | C | M) | (U & C & ~M);    // Stolen
```

Lower-case r, n, d, t and u are used because a seven-segment display cannot produce convincing upper-case R, N, D, T or V; the pattern for u doubles as V. All constants are written directly in the active-low form so the top level can wire them straight to the displays. Because

the display module and the detector are both instantiated and driven by the same U, P and C, the board shows the product name and the two status LEDs at the same time; the simulation confirms that the LED sequence is identical to that of Task 2.2, proving that the two modules do not interfere.

The six products are Couch (000, expensive), Pen (001), Cup (011), Pearl (100, expensive), HDTV (101, expensive) and Cap (110), displayed as COUCH, PEn, CUP, PEArL, Hdtu and CAP respectively; codes 010 and 111 are unused and fall to the default branch.

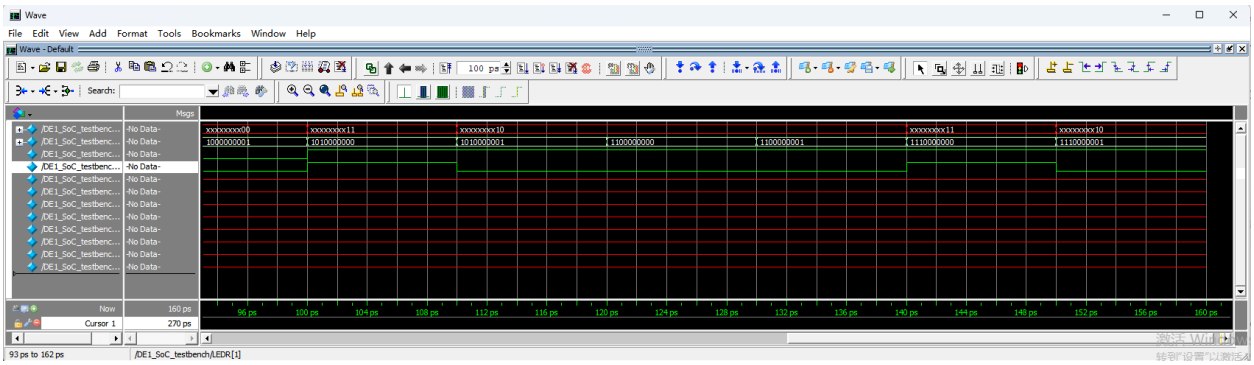


Figure 6. ModelSim waveform of the integrated design (product display plus Discounted and Stolen LEDs).

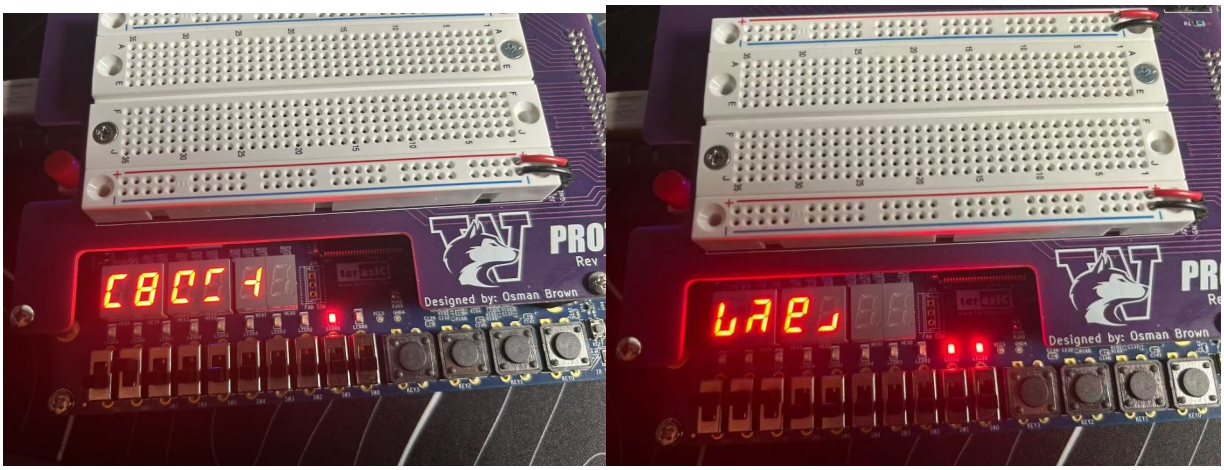


Figure 7. Board photographs of the two unused UPC codes (010 and 111), recording the don't-care patterns actually produced by the hardware.

Discussion

3.1 Challenges encountered and how they were addressed

Problem	Root cause	Solution
Quartus froze, showed a blank white window and every menu item (Save As, right-click → Remove) silently did nothing.	The project was stored under a directory containing Chinese characters, which Quartus 17.0 handles poorly; its file dialogs	Moved the entire project to a pure ASCII path. After that both Quartus and ModelSim behaved normally.

	stopped resolving the path.	
After programming, flipping SW5 and SW1 did not light LEDR[0].	Programmer was started while compilation was still running, so the previously generated .sof was downloaded instead of the new design.	Waited for "Full Compilation was successful" and for the .sof timestamp to update before re-opening the .cdf and pressing Start.
A garbled absolute path pointing at a deleted folder appeared in the project file and could not be removed from the GUI.	The file had been added by browsing, and Quartus stored a relative path whose Chinese folder name was written with a broken encoding.	Deleted the offending line from the .qsf with a byte-level script; Quartus had to be fully closed first, otherwise it rewrites the file on exit.
A leftover quartus.exe process survived the exit and overwrote manual edits.	The crashed GUI left a child process running.	Ended the process in Task Manager before editing any project file.
The first "do runlab.do" stopped with "cannot open macro file *_wave.do" and no waveform appeared.	The *_wave.do file does not exist until the user saves the waveform layout, so the script aborts before reaching "run -all".	Typed run -all manually, saved the layout with File → Save Format, then re-ran do runlab.do.
The waveform window looked empty immediately after loading.	The saved layout zooms to the first nanosecond only.	Used Zoom Full, and zoomed into a narrow window when a single short pulse had to be inspected.
The wave pane disappeared after being closed.	The pane was simply closed; the signal list still exists in the session.	Typed view wave in the Transcript.
ModelSim would not start from the supplied batch file.	Launch_ModelSim.bat hard-codes D:\intelFPGA_lite\17.0\…, but ModelSim was installed at D:\App\FPGA\ModelSim\…	Edited the single path line in the working copy of the batch file.
The simulated LED pattern did not match the expected number of pulses.	Don't-care codes are resolved by the synthesis tool, so cells marked X can legitimately come out as 1.	Read the actual values out of the simulator for all 16 combinations instead of predicting them.

3.2 Lessons learned

• Always simulate before mapping to the FPGA — a design that is wrong in ModelSim will be wrong on the board, and debugging on hardware is far slower.

• Keep Quartus projects on an ASCII-only path; this single change removed almost all of the tool instability encountered in this lab.

• Never press Start in the Programmer until compilation has completely finished and the .sof has been rewritten; a "successful" download does not guarantee the newest design was loaded.

• Don't-care conditions are not "nothing" — they are resolved by the synthesis tool into fixed, often surprising values that can only be found by looking at the real hardware.

• The seven-segment displays are active-low, so a decoder written with 1 = lit must be inverted at the top level.

• Reading bus values with the cursor is far faster than eyeballing a dense waveform when checking whether one narrow pulse occurred.

3.3 Individual work

This lab was completed individually. All five tasks — design, simulation, implementation on the board, waveform capture and this report — were carried out by the author alone, so the hours reported below are for one person.

3.4 Total Number of hours Spent

[16] hours.

References

- Intel. Quartus Prime Lite Edition Design Software, Version 17.0.
- Mentor Graphics / Intel. ModelSim - Intel FPGA Starter Edition, Version 10.5b.
- Terasic Technologies. DE1-SoC Development Board (Cyclone V 5CSEMA5F31C6).
- EE P 535, University of Washington. Lab 1 — An Introduction to Verilog and Digital Logic with FPGA, Autumn 2026.
- EE P 535, University of Washington. Lab 1 Briefing — Step-by-Step Walkthrough, Autumn 2026.
- EE P 535, University of Washington. Quartus Tutorial (Sections 1–6 for tool familiarisation, Section 7 for Task 1.3).
- Hauck, S. Verilog Tutorial. University of Washington, last revised 7/9/20.